

SSL Certificate Monitor - Technical Report

Project: SSL/TLS Certificate Security Monitoring System

Target: Midwest Universities (.edu domains)

Author: Leonardo Ribas

Course: CSCA 5028 - Final Project

Introduction

The SSL Certificate Monitor is a key part of a complete cyber familiarity software that seeks to prevent data breaches as well as to secure the network from any unauthorized accesses in higher education institution. As universities have increasingly come to depend on digital services for teaching, research and administration, the security of SSL/TLS certificate security is now crucial to safeguarding sensitive educational material and preserving institutional confidence.

Problem Statement

Midwestern Universities operate hundreds of web services, all of which need to have valid SSL certificates to prevent eavesdropping. But handling certificate management manually at scale is very challenging:

- **Certificate Expiry:** The expiration of the certificates takes down essential services like student portals, research systems, and administrative platforms
- **Security threats:** Institutions are vulnerable to attacks due usage of weak cryptographic algorithms (SHA-1, small key sizes)
- **Lack of Visibility:** It is hard for IT teams to keep track of everything across a university's range of spaces
- **Reactive Administration:** Problems are frequently found only after services are disrupted for students and faculty.

Solution Overview

The SSL Certificate Monitor delivers university SSL certificates with ****proactive, automated**** security monitoring by:

Auto Discovery & Scanning: It regularly checks SSL certificates on all university domains (.edu) for SSL certificate validity status And provides you with detailed information about this scan. (.edu) is being heavily scoured by 24 hour scans in order to provide complete area coverage.

Smart Security Analysis: Uses common security standards to measure certificate strength, identifying weak algorithms, small key sizes and expiring certificates.

Real-Time Dashboard: Shows IT administrators a single view of the health of the certificates across all their systems, allowing them to quickly spot and prioritize security issues.

Proactive Alerting: Creates real-time alerts for expired, expiring and weak certificates via different mediums (email, webhooks, console alerts).

Scalable architecture: Implemented under microservice architecture for monitoring of hundreds of universities at scale with high availability, performance.

Target Users

- **University IT Security Teams:** Primary users requiring comprehensive certificate oversight
- **System Administrators:** Personnel responsible for maintaining web service availability
- **Compliance Officers:** Staff ensuring adherence to security standards and regulations
- **Regional IT Coordinators:** Personnel overseeing multiple university systems

System Scope

The system focuses specifically on **Midwest university infrastructure**, targeting major public and private institutions across Illinois, Michigan, Ohio, Indiana, Wisconsin, Minnesota, Iowa, and Missouri. This regional approach enables:

- **Tailored Monitoring:** Addresses specific Midwest university consortium requirements
- **Resource Optimization:** Concentrated focus allows for deeper analysis and better service
- **Collaborative Security:** Facilitates information sharing among regional institutions
- **Scalable Foundation:** Architecture designed to expand to additional regions

The SSL Certificate Monitor transforms reactive certificate management into a proactive security capability, helping Midwest universities maintain robust cybersecurity postures while ensuring uninterrupted access to critical digital services.

1. System Requirements

1.1 Functional Requirements

- **Certificate Scanning:** Automatically scan SSL certificates for Midwest university domains every 24 hours
- **Security Analysis:** Analyze certificate security (algorithms, key sizes, expiration) and assign grades A-F
- **Real-time Dashboard:** Single-screen interface showing certificate status, security grades, and alerts
- **Alert System:** Generate notifications for expired/expiring certificates and security vulnerabilities
- **University Management:** Dynamic add/remove/update universities via database instead of hardcoded lists
- **Manual Scanning:** Allow users to trigger immediate certificate scans for specific domains

1.2 Non-Functional Requirements

- **Performance:** Process certificate scans within 5 seconds, dashboard loads under 2 seconds

- **Scalability:** Support 500+ universities with horizontal service scaling
 - **Reliability:** 99.9% uptime with automatic failure recovery
 - **Security:** HTTPS/TLS communication, input validation, CORS protection
 - **Maintainability:** Independent service deployment, comprehensive logging, 80%+ test coverage
-

2. Architecture Design Decisions

2.1 Microservices Architecture

Decision: Implement 4 separate services (Collection, Analysis, Alert, API) + React frontend

- **Separation of Concerns:** Each service has single responsibility
- **Independent Scaling:** Scale services based on individual load
- **Fault Isolation:** Service failures don't affect entire system
- **Team Development:** Services can be developed independently

Trade-off: More complex than monolithic but provides better scalability and maintainability

2.2 Event-Driven Communication

Decision: Redis message queues for inter-service communication

Implementation:

```
Collection Service → scan_queue → Collection Service (processes scans)
Collection Service → analysis_queue → Analysis Service
Analysis Service → alert_queue → Alert Service
```

- **Loose Coupling:** Services don't directly call each other
- **Resilience:** Messages persist if services are down
- **Scalability:** Multiple workers can process same queue

Trade-off: Adds complexity but eliminates cascading failures

2.3 Data Storage Strategy

Decision: OpenSearch for primary storage + Redis for queues/caching

OpenSearch Usage:

- Certificate data with full-text search capabilities
- Alert storage with aggregation for dashboard statistics
- University management with rich metadata

- **Search & Analytics:** Built-in aggregations for dashboard stats
- **JSON Native:** Natural fit for certificate data structure
- **Scalability:** Horizontal scaling for large datasets

2.4 Security Analysis Algorithm

Decision: Multi-factor scoring system (0-100 points) converted to letter grades

Scoring Factors:

```
let score = 100;
if (daysUntilExpiry < 0) score -= 50;
else if (daysUntilExpiry <= 30) score -= 15;
if (signatureAlgorithm.includes('SHA1')) score -= 25; // Weak algorithm
if (keySize < 2048) score -= 25;
```

2.5 Frontend Design

Decision: Single-page React dashboard with 30-second auto-refresh

Components:

- Statistics overview (total/valid/expiring/expired counts)
- University list with status badges and security grades
- Certificate detail panel for selected university
- Recent alerts display

Monitoring dashboards work best with real-time, single-screen design

2.6 Service Responsibilities

Service	Responsibility	Key Functions
Collection	Certificate scanning	TLS connections, data extraction, scheduling
Analysis	Security evaluation	Vulnerability detection, grading, alert generation
Alert	Notification management	Email/webhook notifications, severity routing
API	Frontend interface	Data aggregation, REST endpoints, manual scans

2.7 Database-Driven University Management

Decision: Store universities in OpenSearch instead of hardcoded arrays

Benefits:

- **Dynamic Updates:** Add/remove universities without code deployment
- **Rich Metadata:** Priority levels, student counts, active/inactive status
- **Smart Scheduling:** Scan high-priority universities first
- **Audit Trail:** Track when universities were added/modified

University Data Model:

```
{  
  "name": "University of Illinois",  
  "domain": "illinois.edu",  
  "state": "Illinois",  
  "scanPriority": "high",  
  "isActive": true,  
  "lastScanDate": "2024-07-24T10:30:00Z"  
}
```

3. Technology Stack

3.1 Frontend Technologies

React + TypeScript

Decision: React with TypeScript for the dashboard frontend

- **Component-Based:** Perfect for dashboard widgets (stats, certificate list, details panel)
- **Real-Time Updates:** Easy state management for live data refreshes every 30 seconds
- **Type Safety:** TypeScript prevents runtime errors with complex certificate data structures
- **Developer Experience:** Excellent debugging tools and self-documenting interfaces

Alternatives Considered: Vue.js (smaller ecosystem), Angular (too heavy for single-screen dashboard)

3.2 Backend Technologies

Node.js + TypeScript

Decision: Node.js with TypeScript for all 4 microservices

- **Async I/O:** Perfect for concurrent certificate scanning (multiple TLS connections)
- **Shared Language:** Same TypeScript interfaces across frontend and backend
- **SSL/TLS Libraries:** Native `tls` module for certificate extraction
- **Microservices:** Lightweight services with fast startup times

Performance Example:

```
// Concurrent certificate scanning
const scanPromises = universities.map(async (university) => {
  return scanDomain(university.domain);
});
const results = await Promise.all(scanPromises);
```

Alternatives Considered: Python (slower startup for microservices), Go (less SSL ecosystem)

3.3 Data Storage

OpenSearch (Primary Database)

Decision: OpenSearch for certificates, alerts, and universities

- **JSON Native:** Certificates are naturally JSON documents
- **Full-Text Search:** Search by domain, issuer, university name
- **Built-in Analytics:** Dashboard statistics in single queries
- **Time-Series:** Efficient historical certificate data storage

Analytics Advantage:

```
// Dashboard stats in one query
GET /certificates/_search
{
  "aggs": {
    "security_distribution": { "terms": { "field": "securityGrade" } },
    "status_counts": { "terms": { "field": "status" } }
  }
}
```

Alternatives Considered: PostgreSQL (complex for search/analytics), MongoDB (less powerful analytics)

Redis (Queues + Caching)

Decision: Redis for message queues and caching

- **Simple Queues:** LPUSH/BRPOP operations perfect for job processing
- **Message Persistence:** Jobs survive service restarts
- **Multiple Workers:** Scale processing across multiple service instances
- **Fast Caching:** In-memory performance for dashboard data

Queue Implementation:

```
// Producer: Queue certificate for analysis
await redis.lpush('analysis_queue', JSON.stringify(certificateData));

// Consumer: Process analysis jobs
const job = await redis.brpop('analysis_queue', 10);
```

Alternatives Considered: RabbitMQ (overkill for simple use case), Database queues (adds load to primary DB)

3.4 Deployment

Docker + Docker Compose

Decision: Containerization with orchestration

- **Environment Consistency:** Same runtime from development to production
- **Service Isolation:** Each microservice in separate container
- **Single Command Setup:** `docker-compose up` starts entire system
- **Cloud Portability:** Runs on Heroku, AWS, or any container platform

Development Benefits:

```
# docker-compose.yml
services:
  collection-service:
    build: ./collection-service
    depends_on: [redis, opensearch]
  redis:
    image: redis:7-alpine
  opensearch:
    image: opensearchproject/opensearch:2.11.1
```

Alternatives Considered: Kubernetes (overkill for development), Virtual machines (resource-heavy)

3.5 Communication

REST APIs + Event-Driven Queues

Decision: RESTful APIs for frontend + Redis queues for service communication

REST API:

- **HTTP Standard:** Leverages existing infrastructure and tooling
- **Simple Integration:** Native browser support, easy debugging
- **Resource-Based:** Clear endpoint design (`/api/certificates`, `/api/universities`)

Event-Driven:

- **Loose Coupling:** Services don't directly depend on each other
- **Fault Tolerance:** System continues if one service fails
- **Scalability:** Multiple workers can process same queue

Message Flow:

```
Collection Service → analysis_queue → Analysis Service → alert_queue → Alert Service
```

Alternatives Considered: GraphQL (too complex for CRUD), Synchronous calls (tight coupling)

3.6 Technology Decision Summary

Technology	Primary Reason	Key Benefit
React + TypeScript	Component UI + Type safety	Real-time dashboard with fewer bugs
Node.js + TypeScript	Async I/O + Shared language	Efficient concurrent certificate scanning
OpenSearch	JSON-native + Analytics	Complex queries and dashboard stats
Redis	Simple queues + Fast caching	Reliable message delivery and performance
Docker	Environment consistency	Same setup development to production
REST + Queues	Standards + Loose coupling	Easy development and fault tolerance

4. Key Benefits

Scalable: Independent service scaling based on load

Maintainable: Clear service boundaries and responsibilities

Reliable: Queue-based fault tolerance and automatic recovery

Flexible: Database-driven configuration without code changes

Real-time: Live dashboard updates every 30 seconds

Actionable: Security grades and recommendations for remediation